

Análisis de patrones de brillo de cielo nocturno

Víctor Tilve

2025-09-05

Primer análisis de los datos.

En primer lugar se indica la carpeta donde se encuentran los archivos en formatos Turtle y RDF/XML para cargarlos. Muchos ya fueron tratados mediante consultas SPARQL con la herramienta GraphDB para llegar aquí con toda la información necesaria y requerida. Después se define la consulta SPARQL cuyo resultado será transformado en un dataframe de R.

```
library(readr)
library(ggplot2)
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(rdflib)

rdf_data <- "/media/victor/1FPB-COTARE/Cousas/TFM-IA/TFM-trabajo/00.CousasDefinitivas/clustering"

#Función para procesar un solo archivo RDF/XML
#Mucho ojo con los prefijos porque en el final / o # debe coincidir con el grafo
procesar_rdf <- function(file_path) {
  rdf <- rdf_parse(file_path, format = "rdfxml")
  consulta <- '
PREFIX sosa: <http://www.w3.org/ns/sosa/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ex: <http://example.org#>

SELECT ?observacion ?sensor ?fecha ?valor ?noche ?luna ?estacion ?finSemana
WHERE {
  ?observacion rdf:type sosa:Observation ;
    sosa:madeBySensor ?sensor ;
    sosa:resultTime ?fecha ;
    sosa:hasSimpleResult ?valor ;
    ex:esNocheAstronomica ?noche ;
    ex:hayBrilloLuna ?luna ;
    ex:perteneceAEstacion ?estacion;
```

```

        ex:finDeSemana ?finSemana .
    }'

resultado <- rdf_query(rdf, consulta)
df <- as.data.frame(resultado)

df$valor <- as.numeric(df$valor)
df$noche <- tolower(df$noche) == "true"
df$luna <- tolower(df$luna) == "true"
df$finSemana <- tolower(df$finSemana) == "true"
df$fecha <- as.POSIXct(df$fecha, format="%Y-%m-%dT%H:%M:%S", tz="UTC")

return(df)
}

```

En esta parte del código se decide si se leen todos los archivos de la carpeta indicada anteriormente o mediante expresiones regulares se ponen las condiciones que se decidan. En este caso se cargan cuatro archivos RDF/XML con el resultado de una consulta de SPARQL que recupera todas las observaciones realizadas con los fotómetros tras su enriquecimiento con información variada: si hay brillo de Luna, estación del año y si es fin de semana o no. Se hace así porque un único archivo pesa más de 350MB y R se reinicia al procesarlo. Sin embargo, dividido en 4 partes sí consigue cargar los datos.

```

# Lista de archivos RDF en la carpeta
archivos <- list.files(rdf_data, pattern = "result04_.*\\.xml$",
                      full.names = TRUE)

# Procesar y combinar todos los data.frames
todos_los_datos <- bind_rows(lapply(archivos, procesar_rdf))

```

Se hace ahora un criba de los datos seleccionando aquellos de interés para el estudio, que son dentro de la noche astronómica y sin Luna (aunque esto ya se hizo en la descarga con SPARQL), y se eliminan los datos atípicos.

```

datos_filtrados <- subset(todos_los_datos, noche == TRUE & luna == FALSE)
rm(todos_los_datos)
datos_filtrados$solo_fecha <- as.Date(datos_filtrados$fecha) # solo la fecha (YYYY-MM-DD)
datos_filtrados$solo_hora <- format(datos_filtrados$fecha, format = "%H:%M:%S") # solo la hora
datos_filtrados <- subset(datos_filtrados, valor > 0)
#Valores atípicos

lim_inf <- quantile(datos_filtrados$valor, 0.01, na.rm = TRUE)
lim_sup <- quantile(datos_filtrados$valor, 0.99, na.rm = TRUE)
datos_filtrados <- subset(datos_filtrados, valor >= lim_inf & valor <= lim_sup)

#Valores atípicos

Q1 <- quantile(datos_filtrados$valor, 0.25, na.rm = TRUE)
Q3 <- quantile(datos_filtrados$valor, 0.75, na.rm = TRUE)
IQR_valor <- Q3 - Q1

lim_inf <- Q1 - 1.5 * IQR_valor
lim_sup <- Q3 + 1.5 * IQR_valor

outliers <- datos_filtrados %>%
  filter(valor < lim_inf | valor > lim_sup)

```

```
datos_filtrados <- subset(datos_filtrados, valor >= lim_inf & valor <= lim_sup)

#Etiqueta
datos_filtrados$etiqueta_sensor <- sub(".*#([_]+[_]+).*", "\\1", datos_filtrados$sensor)
```

Para las horas se toman en formato cíclico para evitar los saltos que hay con el cambio de día. Para documentarse sobre la codificación cíclica horaria se pueden consultar los enlaces:

<https://www.kaggle.com/code/avanwyk/encoding-cyclical-features-for-deep-learning>

<https://medium.com/@axelazara6/why-we-need-encoding-cyclical-features-79ecc3531232>

```
# Codificar la hora decimal como variable circular
datos_filtrados <- datos_filtrados %>%
  mutate(
    hora_decimal = as.numeric(format(fecha, "%H")) + as.numeric(format(fecha, "%M")) / 60,
    hora_minuto = format(fecha, "%H:%M")
  )

datos_filtrados$hora_sin <- sin(2 * pi * datos_filtrados$hora_decimal / 24)
datos_filtrados$hora_cos <- cos(2 * pi * datos_filtrados$hora_decimal / 24)
```

Patrón fin de semana

En este apartado se va a analizar si existe un patrón de fin de semana.

```
# Paquetes
library(dplyr)
library(ggplot2)

# ----- Boxplots DS vs FS para cada sensor -----
sensor_col = "etiqueta_sensor"
value_col = "valor"
time_col = "fecha"
finDeSemana_col = "finSemana"

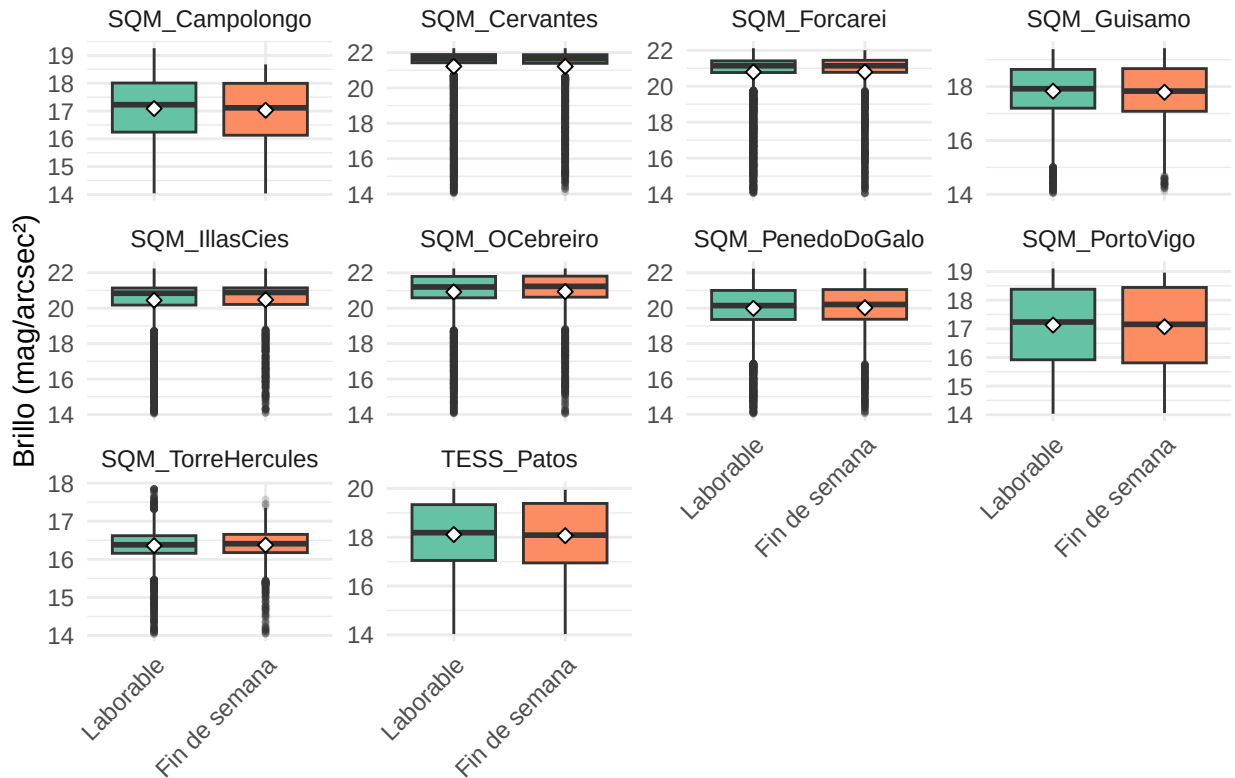
# Etiquetas bonitas
datos_filtrados$fin_semana_lbl <- factor(ifelse(datos_filtrados$finSemana,
                                              "Fin de semana", "Laborable"),
                                         levels = c("Laborable", "Fin de semana"))

# Gráfico facetado por sensor
g <- ggplot(datos_filtrados, aes(x = fin_semana_lbl,
                                y = .data[[value_col]],
                                fill = fin_semana_lbl)) +
  geom_boxplot(outlier.alpha = 0.2, outlier.size = 0.7) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 1.8,
              fill = "white") +
  facet_wrap(vars(.data[[sensor_col]]), scales = "free_y") +
  scale_fill_brewer(palette = "Set2", guide = "none") +
  labs(title = "Distribución del brillo por sensor (fin de semana vs laborable)",
       x = NULL, y = "Brillo (mag/arcsec²)") +
  theme_minimal(base_size = 11) +
```

```
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

g

Distribución del brillo por sensor (fin de semana vs laborable)



Parece que no hay diferencias entre días laborables y fin de semana en este análisis global. Se muestra a continuación una tabla con los datos ordenados para salir de dudas:

```
library(dplyr)
library(tidyr)
library(knitr)

# Calcula estadísticos
stats_long <- datos_filtrados %>%
  filter(!is.na(.data[[value_col]])) %>%
  group_by(!sym(sensor_col), fin_semana_lbl) %>%
  summarise(
    n      = dplyr::n(),
    media  = mean(.data[[value_col]], na.rm = TRUE),
    mediana= median(.data[[value_col]], na.rm = TRUE),
    sd     = sd(.data[[value_col]], na.rm = TRUE),
    q1     = quantile(.data[[value_col]], 0.25, na.rm = TRUE, type = 7),
    q3     = quantile(.data[[value_col]], 0.75, na.rm = TRUE, type = 7),
    .groups = "drop"
  ) %>%
  arrange(!sym(sensor_col), fin_semana_lbl)
```

```
stats_long
```

```
## # A tibble: 20 x 8
##   etiqueta_sensor fin_semana_lbl      n media mediana    sd    q1    q3
##   <chr>           <fct>         <int> <dbl>  <dbl> <dbl> <dbl> <dbl>
## 1 SQM_Campolongo Laborable    12854  17.1   17.2  1.02  16.2  18.0
## 2 SQM_Campolongo Fin de semana 5031  17.0   17.1  1.03  16.1  18.0
## 3 SQM_Cervantes Laborable    11222  21.2   21.6  1.46  21.4  21.9
## 4 SQM_Cervantes Fin de semana 4365  21.2   21.6  1.43  21.4  21.9
## 5 SQM_Forcarei Laborable    12637  20.8   21.1  1.25  20.8  21.4
## 6 SQM_Forcarei Fin de semana 4938  20.8   21.1  1.27  20.8  21.4
## 7 SQM_Guisamo Laborable    12741  17.8   17.9  0.926 17.2  18.6
## 8 SQM_Guisamo Fin de semana 4948  17.8   17.8  0.970 17.1  18.7
## 9 SQM_IllasCies Laborable    12663  20.4   20.8  1.23  20.2  21.1
## 10 SQM_IllasCies Fin de semana 4951  20.5   20.9  1.22  20.2  21.1
## 11 SQM_OCebreiro Laborable    12495  20.9   21.2  1.33  20.6  21.8
## 12 SQM_OCebreiro Fin de semana 4870  20.9   21.2  1.34  20.6  21.8
## 13 SQM_PenedoDoGalo Laborable    12730  20.0   20.1  1.28  19.4  21.0
## 14 SQM_PenedoDoGalo Fin de semana 4989  20.0   20.2  1.30  19.4  21.1
## 15 SQM_PortoVigo Laborable    12238  17.1   17.2  1.31  15.9  18.4
## 16 SQM_PortoVigo Fin de semana 4783  17.1   17.2  1.35  15.8  18.4
## 17 SQM_TorreHercules Laborable    12857  16.4   16.4  0.415 16.2  16.6
## 18 SQM_TorreHercules Fin de semana 5047  16.4   16.4  0.418 16.2  16.7
## 19 TESS_Patos Laborable    111962  18.1   18.2  1.29  17.0  19.3
## 20 TESS_Patos Fin de semana 42400  18.1   18.1  1.33  17.0  19.4
```

Efectivamente, no hay ningún cambio significativo entre los días de semana y fines de semana.

Patrón estaciones

A continuación se repiten los cálculos anteriores pero considerando las estaciones del año:

```
# Gráfico facetado por sensor
map <- setNames(
  c("Invierno", "Primavera", "Otoño", "Verano"),
  c("http://example.org#Invierno",
    "http://example.org#Primavera",
    "http://example.org#Otono",      # ojo: en la URI es Otono (sin ñ)
    "http://example.org#Verano")
)

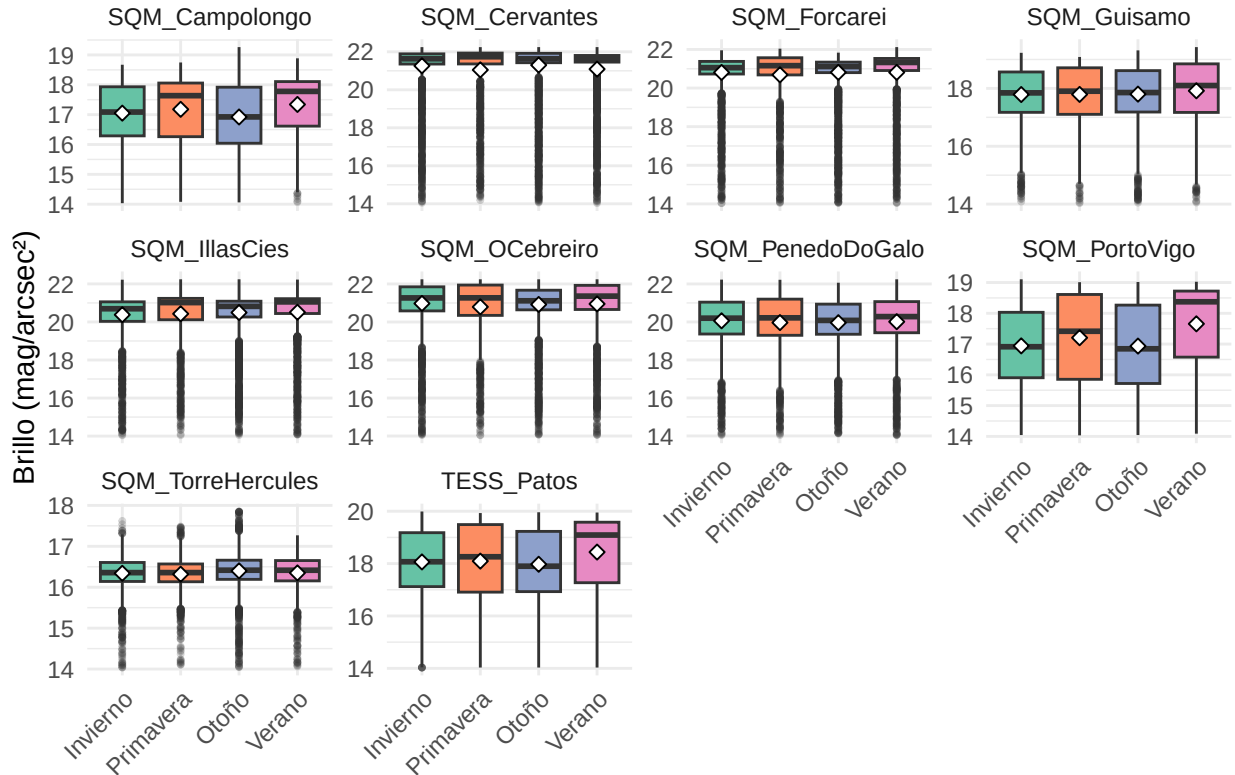
# Aplicar el mapeo y crear factor ordenado
datos_filtrados$estaciones <- map[as.character(datos_filtrados$estacion)]
datos_filtrados$estaciones <- factor(
  datos_filtrados$estaciones,
  levels = c("Invierno", "Primavera", "Otoño", "Verano")
)

g <- ggplot(datos_filtrados, aes(x = estaciones,
  y = .data[[value_col]],
  fill = estaciones)) +
  geom_boxplot(outlier.alpha = 0.2, outlier.size = 0.7) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 1.8,
    fill = "white") +
```

```
facet_wrap(vars(.data[[sensor_col]]), scales = "free_y") +
scale_fill_brewer(palette = "Set2", guide = "none") +
labs(title = "Distribución del brillo por sensor (fin de semana vs laborable)",
x = NULL, y = "Brillo (mag/arcsec²)") +
theme_minimal(base_size = 11)+
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

g

Distribución del brillo por sensor (fin de semana vs laborable)



En este caso sí parece haber diferencias evidentes en algunos casos, especialmente en verano, donde parece que la oscuridad aumenta en los lugares más urbanizados.

Análisis de periodos

Por último se va a ver el uso de grafos para el estudio de periodos determinados. En este caso se realiza un análisis de los datos de los fotómetros SQM de Cervantes, Forcarei, Porto de Vigo, Illas Cíes y Penedo do Galo en el último trimestre de 2023, los cuatro trimestres de 2024 y enero de 2025. Se comienza como en todos los casos leyendo el grafo:

```
library(rdfliib)
library(dplyr)
library(tidyr)

# === 1) Cargar el grafo ===
ttl_path <- "/media/victor/1FPB-COTARE/Cousas/TFM-IA/TFM-trabajo/00.CousasDefinitivas/clustering/boxpl
```

```
g <- rdf_parse(ttl_path, format = "turtle")
```

Se define y aplica sobre el grafo la consulta SPARQL para obtener los datos de interés:

```
# === 2) Consulta SPARQL: sensor, intervalo y estadísticos ===
q_stats <- 'PREFIX STATO: <http://purl.obolibrary.org/obo/>
PREFIX dcterms: <http://purl.org/dc/terms/>
PREFIX sosa: <http://www.w3.org/ns/sosa/>
PREFIX prov: <http://www.w3.org/ns/prov#>
PREFIX time: <http://www.w3.org/2006/time#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT ?sensor ?stat ?statType ?statLabel ?value ?interval ?start ?end ?source
WHERE {

  {
    # Caso A: el estadístico es parte de un boxplot
    ?box a STATO:STATO_0000243 ;
        dcterms:subject ?sensor ;
        dcterms:hasPart ?stat .
    ?stat a ?statType ;
        rdf:value ?value ;
        prov:wasDerivedFrom ?ds .
    ?ds sosa:phenomenonTime ?interval .
    BIND(?box AS ?source)
  }
  UNION
  {
    # Caso B: el estadístico está "suelto"
    ?stat a ?statType ;
        rdf:value ?value .
    BIND(?stat AS ?source)
    OPTIONAL { ?stat dcterms:subject ?sensor }
    OPTIONAL { ?stat prov:wasDerivedFrom ?ds }
    OPTIONAL { ?stat sosa:phenomenonTime ?interval }
    OPTIONAL { ?ds dcterms:subject ?sensor }
    OPTIONAL { ?ds sosa:madeBySensor ?sensor }
  }

  # Limitar a los tipos STATO que te interesan
  FILTER (?statType IN (
    STATO:STATO_0000164, # IQR
    STATO:STATO_0000167, # Q1
    STATO:STATO_0000574, # Mediana
    STATO:STATO_0000170, # Q3
    STATO:STATO_0000150, # Min
    STATO:STATO_0000151, # Max
    STATO:STATO_0000573, # Media
    STATO:STATO_0000237, # DesvTip
    STATO:STATO_0000033 # Moda
  ))

  # Inicio/fin del intervalo (sin property paths)
```

```

OPTIONAL { ?interval time:hasBeginning ?b1 . ?b1 time:inXSDDateTime ?startDT1 . }
OPTIONAL { ?interval time:hasBeginning ?b2 . ?b2 time:inXSDDateTimeStamp ?startDT2 . }
OPTIONAL { ?interval time:hasEnd ?e1 . ?e1 time:inXSDDateTime ?endDT1 . }
OPTIONAL { ?interval time:hasEnd ?e2 . ?e2 time:inXSDDateTimeStamp ?endDT2 . }

BIND(COALESCE(?startDT1, ?startDT2) AS ?start)
BIND(COALESCE(?endDT1, ?endDT2) AS ?end)

# Etiquetas legibles (opcional)
OPTIONAL {
  VALUES (?statType ?statLabel) {
    (STATO:STATO_0000164 "IQR")
    (STATO:STATO_0000167 "Q1")
    (STATO:STATO_0000574 "Mediana")
    (STATO:STATO_0000170 "Q3")
    (STATO:STATO_0000150 "Min")
    (STATO:STATO_0000151 "Max")
    (STATO:STATO_0000573 "Media")
    (STATO:STATO_0000237 "DesvTip")
    (STATO:STATO_0000033 "Moda")
  }
}
}
ORDER BY ?sensor ?interval ?statLabel

'

res <- rdf_query(g, q_stats) %>%
  mutate(
    value = suppressWarnings(as.numeric(value)),
    start = as.POSIXct(start, tz = "UTC"),
    end = as.POSIXct(end, tz = "UTC"),
    statLabel = ifelse(is.na(statLabel) | statLabel == "", as.character(statType), statLabel)
  )

```

Ahora se representan los datos que estaban en el grafo en forma de diagrama de barras:

```

library(dplyr)
library(ggplot2)
library(stringr)
library(forcats)

# res: data.frame con columnas: sensor, interval, start, end, statLabel, value
df_plot <- res %>%
  mutate(
    value = suppressWarnings(as.numeric(value)),

    # Local names (parte tras # o /)
    sensor_local = sub("^.*[#/]", "", as.character(sensor)),
    interval_local = sub("^.*[#/]", "", as.character(interval)),

    # Sensor corto: primeros dos segmentos separados por "_"
    sensor_label = ifelse(

```



```

    grepl("^[^_]+_[^_]+$", sensor_local),
    sub("^[^_]+_[^_]+$.*", "\\1", sensor_local), # captura A_B y descarta el resto
    sensor_local
  ),

  # Intervalo corto: segundo segmento tras "intervalo_"
  interval_label = dplyr::case_when(
    !is.na(interval_local) & grepl("^intervalo_", interval_local) ~
      sub("^intervalo_(^[^_]+).*", "\\1", interval_local), # queda p.ej. Trim12024
    TRUE ~ interval_local
  )
) %>%
# Quitar Desviación típica e IQR
filter(!is.na(value),
  statLabel %in% c("Media", "Mediana", "Min", "Q1", "Q3", "Max", "Moda")) %>%
# Agregar por si hay duplicados en sensor-intervalo-estadístico
group_by(sensor_label, interval_label, statLabel) %>%
summarise(value = mean(value, na.rm = TRUE), .groups = "drop")

# Orden bonito de las medidas
stat_order <- c("Min", "Q1", "Mediana", "Media", "Q3", "Max", "Moda")
df_plot <- df_plot %>% mutate(statLabel = factor(statLabel, levels = stat_order))

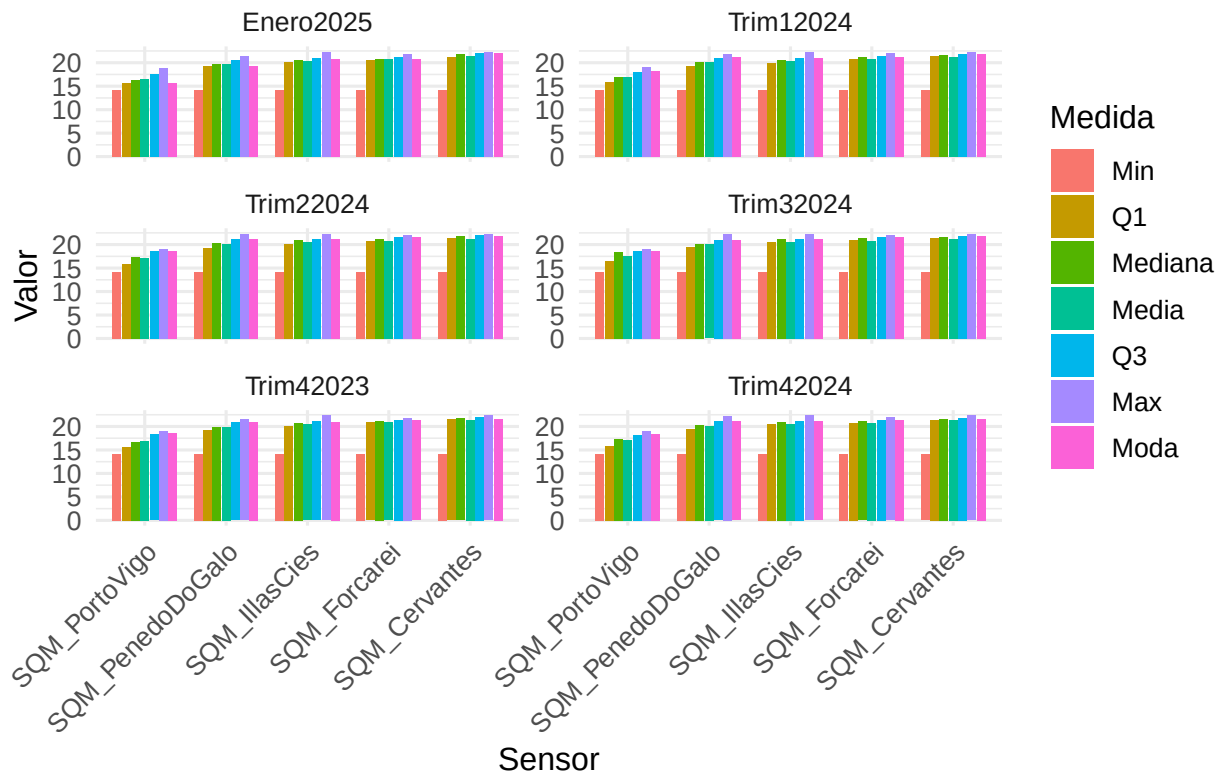
# Ordenar sensores por el promedio de Media/Mediana (opcional)
sensor_order <- df_plot %>%
  filter(statLabel %in% c("Mediana", "Media")) %>%
  group_by(sensor_label) %>%
  summarise(aux = mean(value, na.rm = TRUE), .groups = "drop") %>%
  arrange(aux) %>% pull(sensor_label)

df_plot <- df_plot %>%
  mutate(sensor_label = factor(sensor_label, levels = sensor_order))

# --- Barras por sensor, facet por intervalo ---
ggplot(df_plot, aes(x = sensor_label, y = value, fill = statLabel)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7) +
  facet_wrap(~ interval_label, ncol = 2, scales = "free_y") +
  labs(
    title = "Estadísticos por sensor e intervalo (sin DesvTip ni IQR)",
    x = "Sensor", y = "Valor", fill = "Medida"
  ) +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

Estadísticos por sensor e intervalo (sin DesvTip ni IQR)



No es fácil apreciar tendencias o patrones en estos diagramas, así que se representan a continuación los valores de los datos estadísticos ordenados por los periodos seleccionados:

```
library(dplyr)
library(tidyr)
library(stringr)
library(ggplot2)
library(purrr)
library(forcats)

# --- Orden explícito del eje X (intervalos) ---
interval_order <- c("Trim42023", "Trim12024", "Trim22024", "Trim32024", "Trim42024", "Enero2025")
# Si en realidad querías "Trim42024", cámbialo aquí ↑

# --- Qué estadísticas dibujar ---
stats_to_plot <- c("Min", "Q1", "Mediana", "Media", "Q3", "Max", "Moda")

# res debe tener: sensor, interval, start, end, statLabel, value
df_plot <- res %>%
  mutate(
    value = suppressWarnings(as.numeric(value)),

    # nombres cortos (tras # o /)
    sensor_local = sub("^.*[#/]", "", as.character(sensor)),
    interval_local = sub("^.*[#/]", "", as.character(interval)),

    # Sensor corto: "SQM_Cervantes" de "SQM_Cervantes_ANC..."
```

```

sensor_label = ifelse(
  grepl("^[^_]+_[^_]+$", sensor_local),
  sub("^(^[^_]+_[^_]+).*", "\\1", sensor_local),
  sensor_local
),

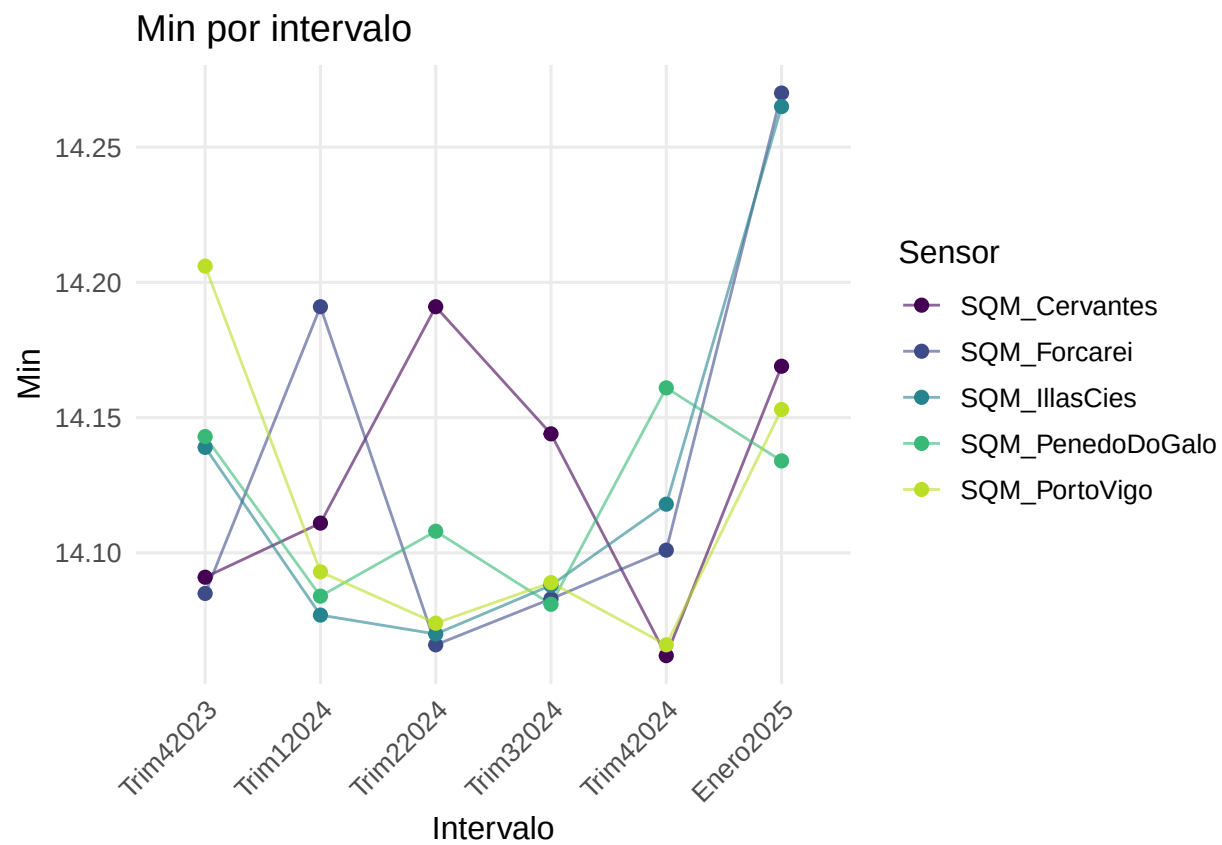
# Intervalo corto: "Trim12024" de "intervalo_Trim12024_..."
interval_label = dplyr::case_when(
  !is.na(interval_local) & grepl("^intervalo_", interval_local) ~
    sub("^intervalo_(^[^_]+).*", "\\1", interval_local),
  TRUE ~ interval_local
)
) %>%
filter(!is.na(value), statLabel %in% stats_to_plot) %>%
group_by(sensor_label, interval_label, statLabel) %>%
summarise(value = mean(value, na.rm = TRUE), .groups = "drop") %>%
mutate(
  interval_label = factor(interval_label, levels = interval_order),
  statLabel = factor(statLabel, levels = c("Min", "Q1", "Mediana", "Media", "Q3", "Max", "Moda"))
)

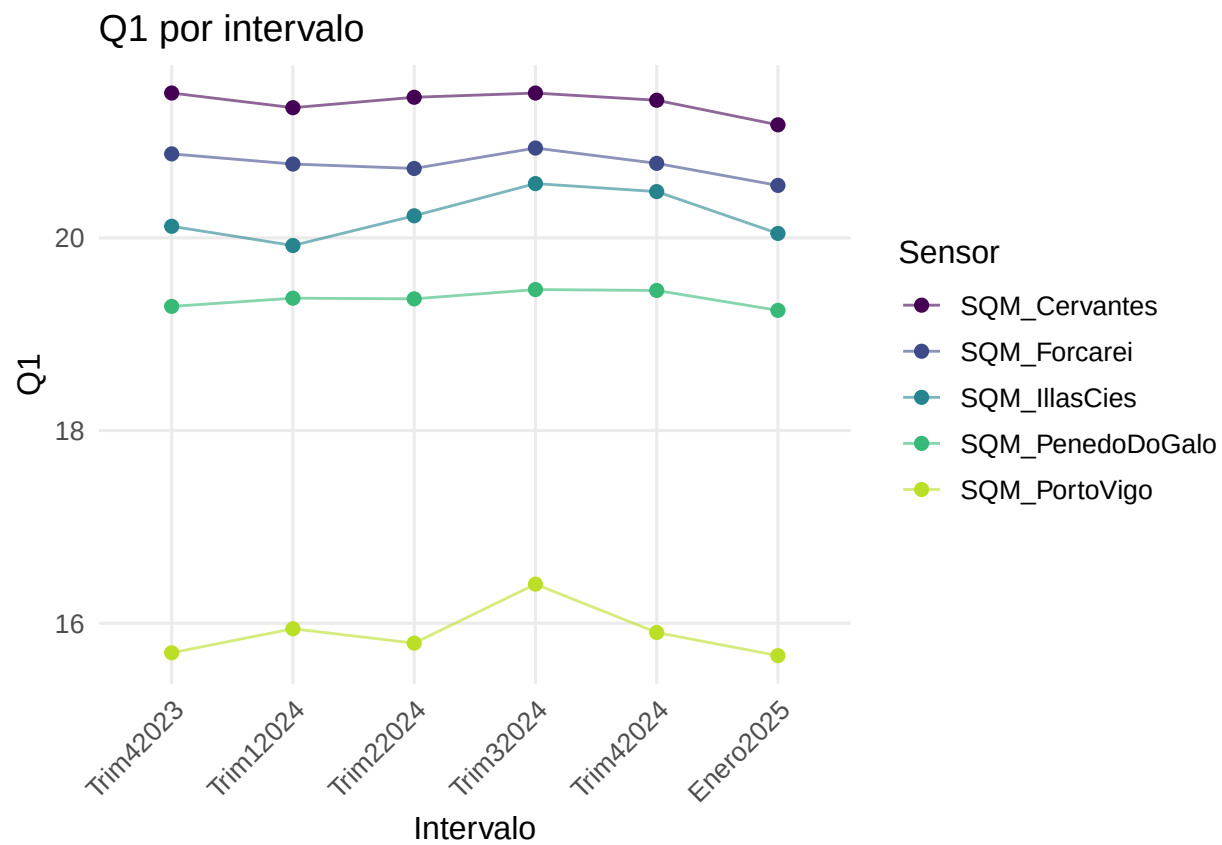
# --- Función para dibujar un estadístico (una figura por 'dato') ---
make_plot <- function(stat_name) {
  ggplot(filter(df_plot, statLabel == stat_name),
    aes(x = interval_label, y = value,
        color = sensor_label, group = sensor_label)) +
  geom_line(alpha = 0.6) +
  geom_point(size = 2) +
  scale_color_viridis_d(option = "D", end = 0.9, name = "Sensor") +
  labs(title = paste0(stat_name, " por intervalo"),
    x = "Intervalo", y = stat_name) +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid.minor = element_blank())
}

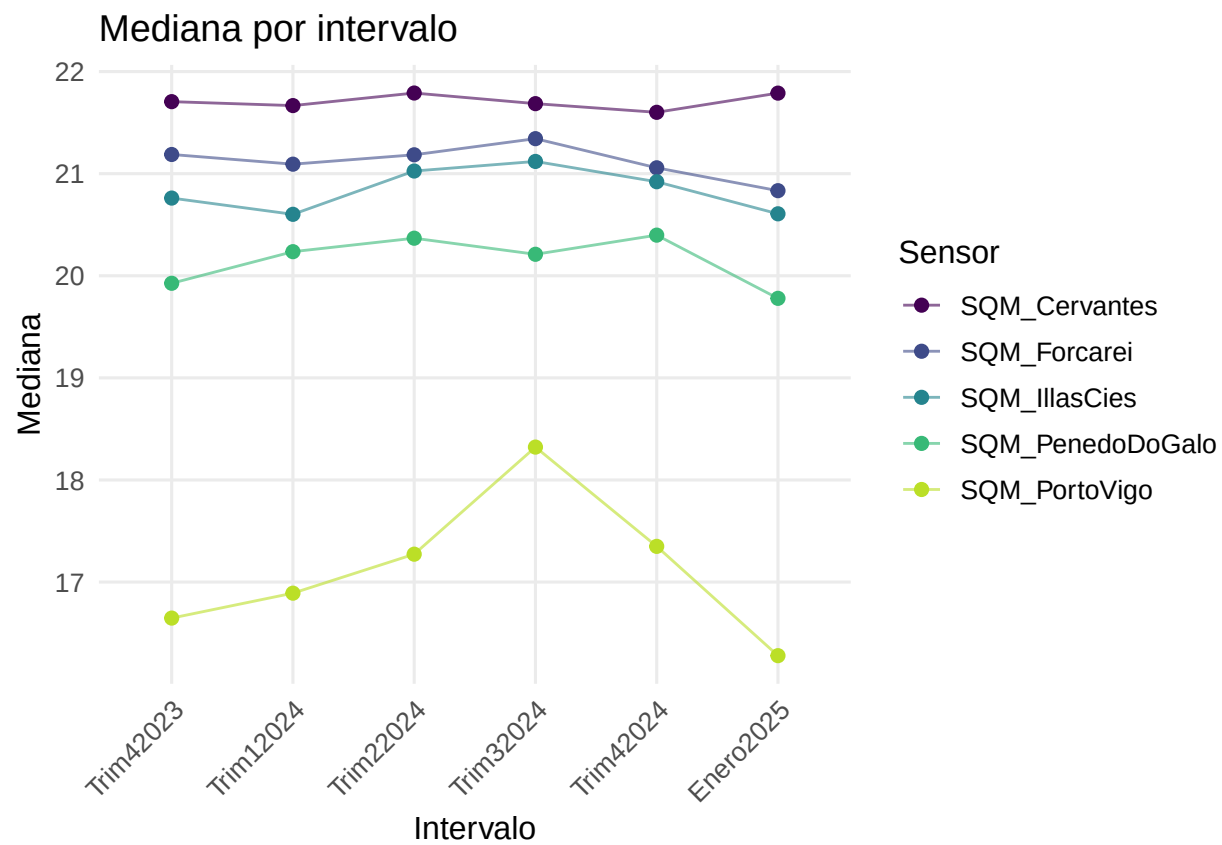
# --- Crear todas las figuras ---
plots <- setNames(lapply(stats_to_plot, make_plot), stats_to_plot)

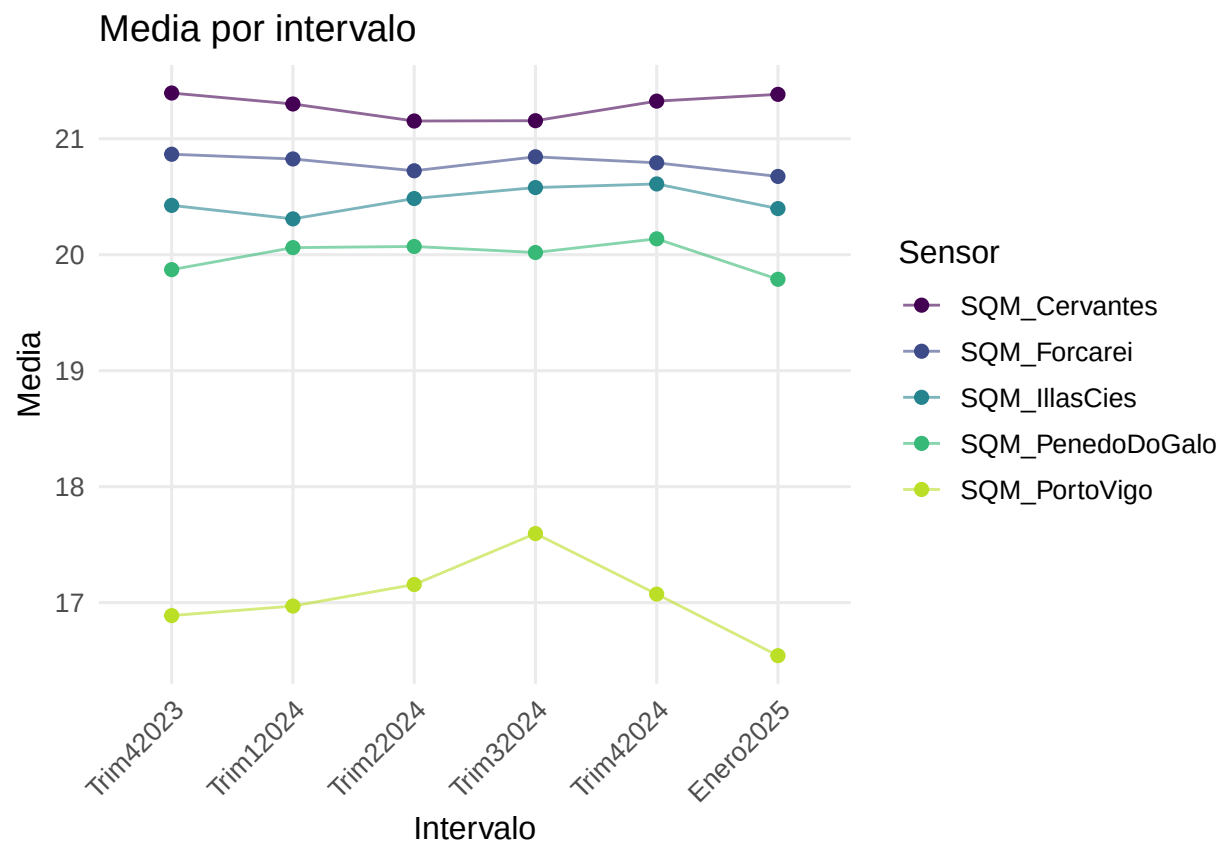
# Mostrar en pantalla
for (nm in names(plots)) print(plots[[nm]])

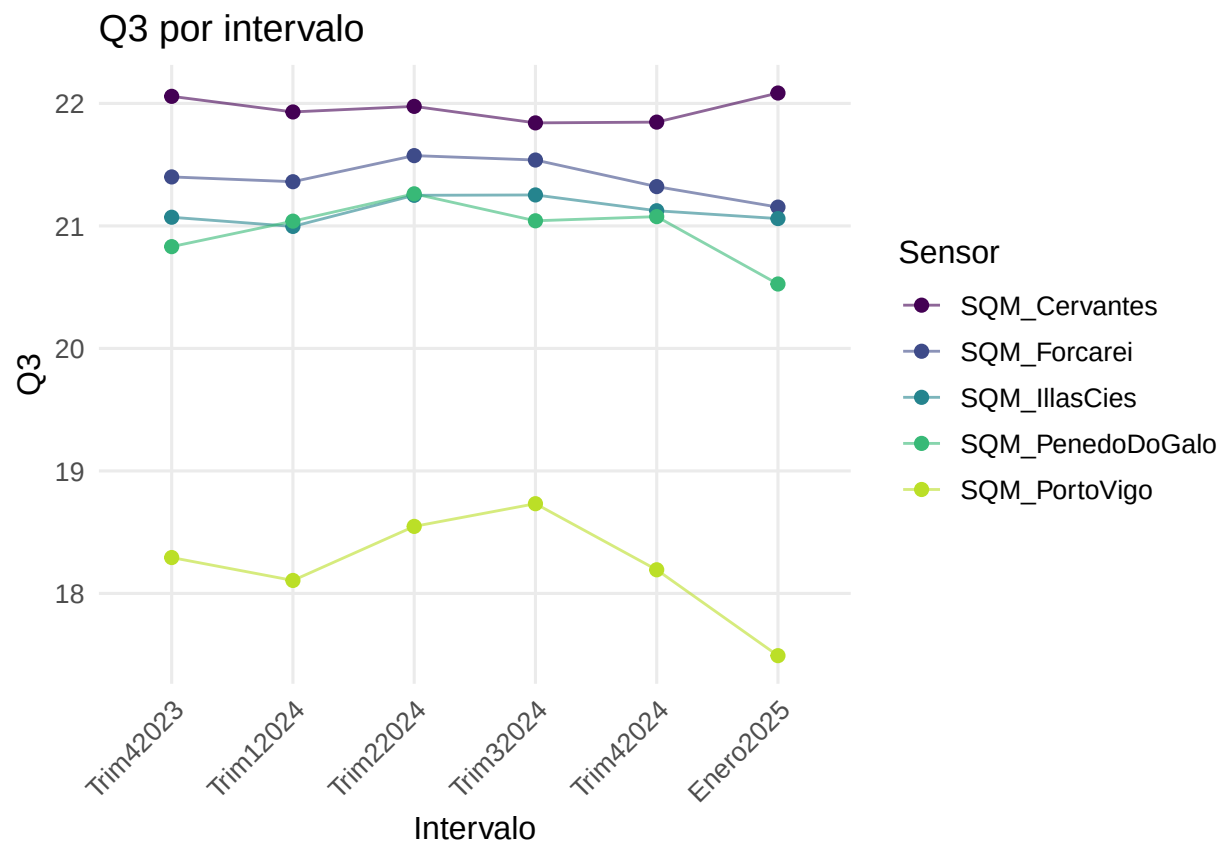
```

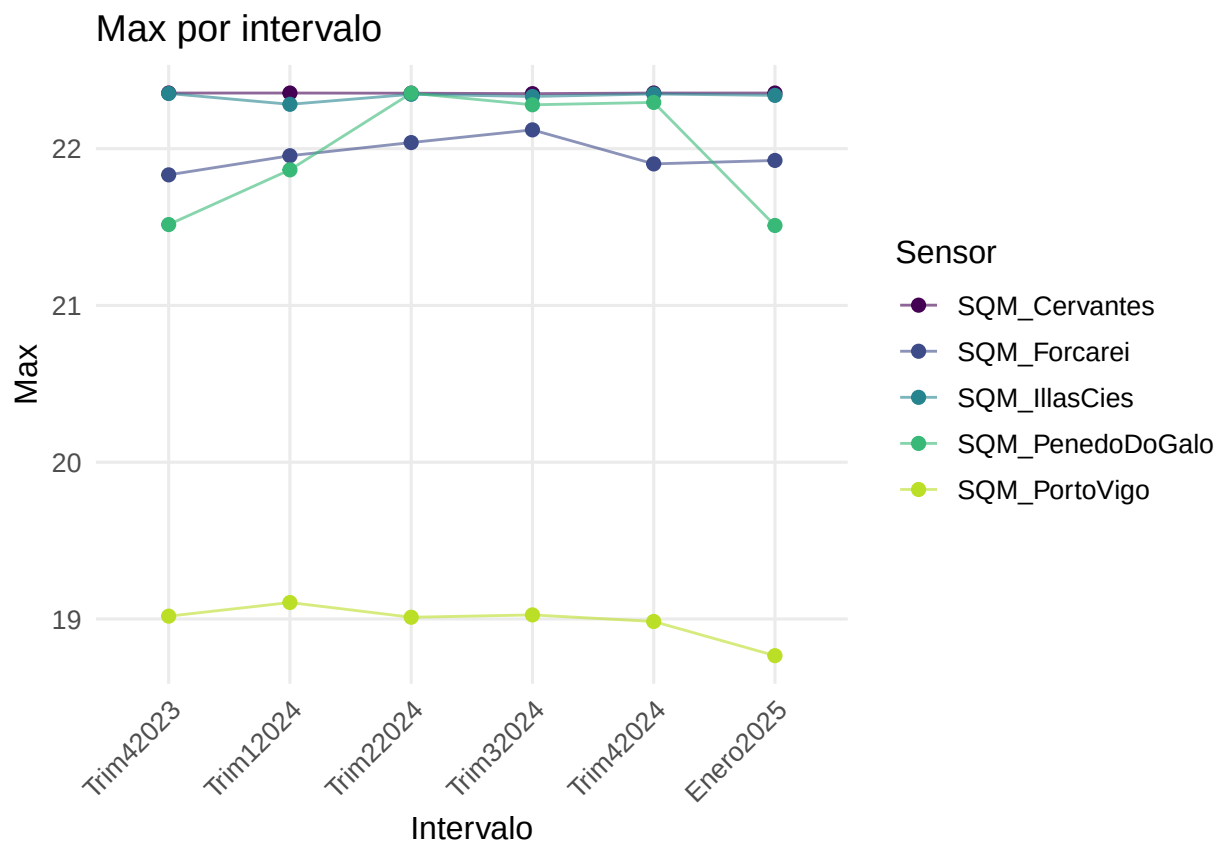


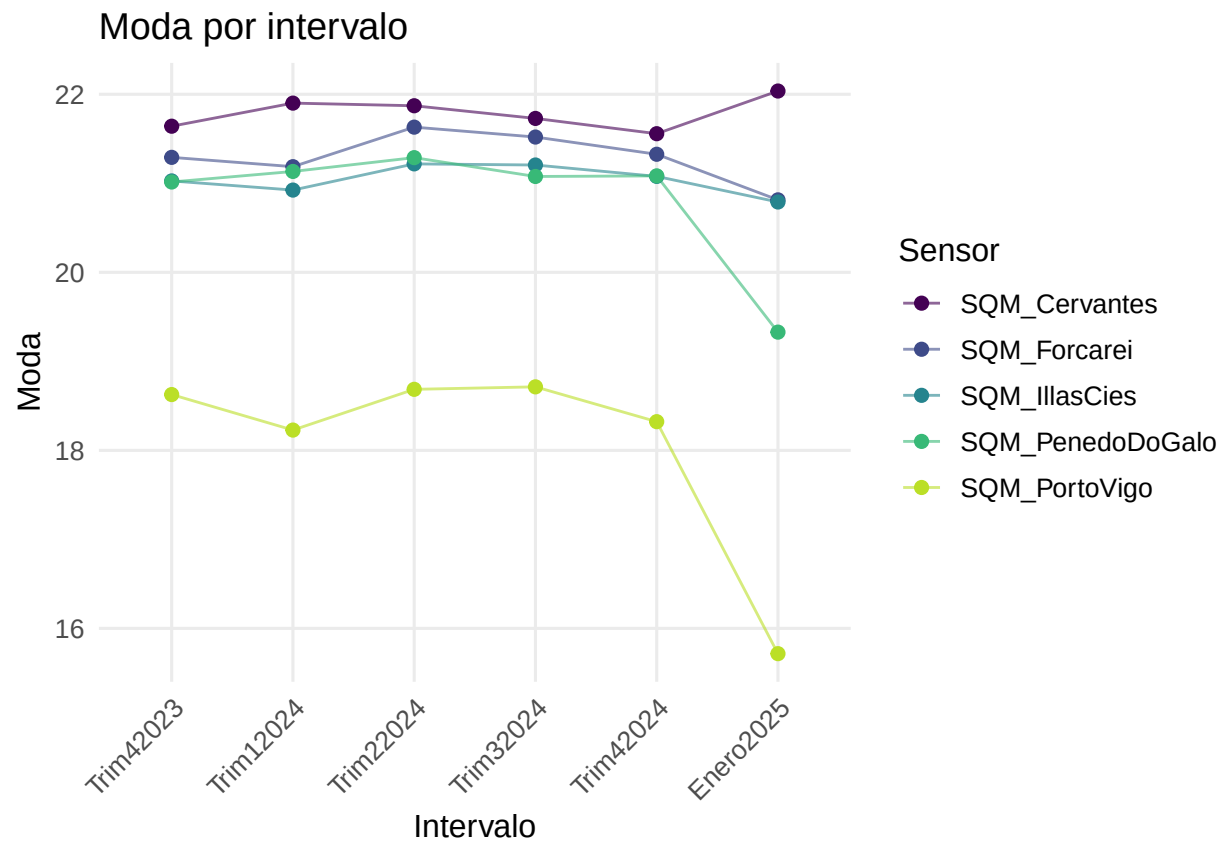












```
# (Opcional) Guardar a disco
# dir.create("plots_stats", showWarnings = FALSE)
# purrr::iwalk(plots, ~ ggsave(file.path("plots_stats", paste0("stat_", .y, ".png")),
#                               .x, width = 10, height = 6, dpi = 150))
#
```

En estas gráficas se aprecia que durante el tercer trimestre de 2024 hubo un cambio en el brillo de cielo importante. Se corresponde con parte del verano, lo que viene confirmando el análisis hecho por estaciones.